Name	Farooq Ahmed
Field	Data Science & Analytics
ID	DHC-5217

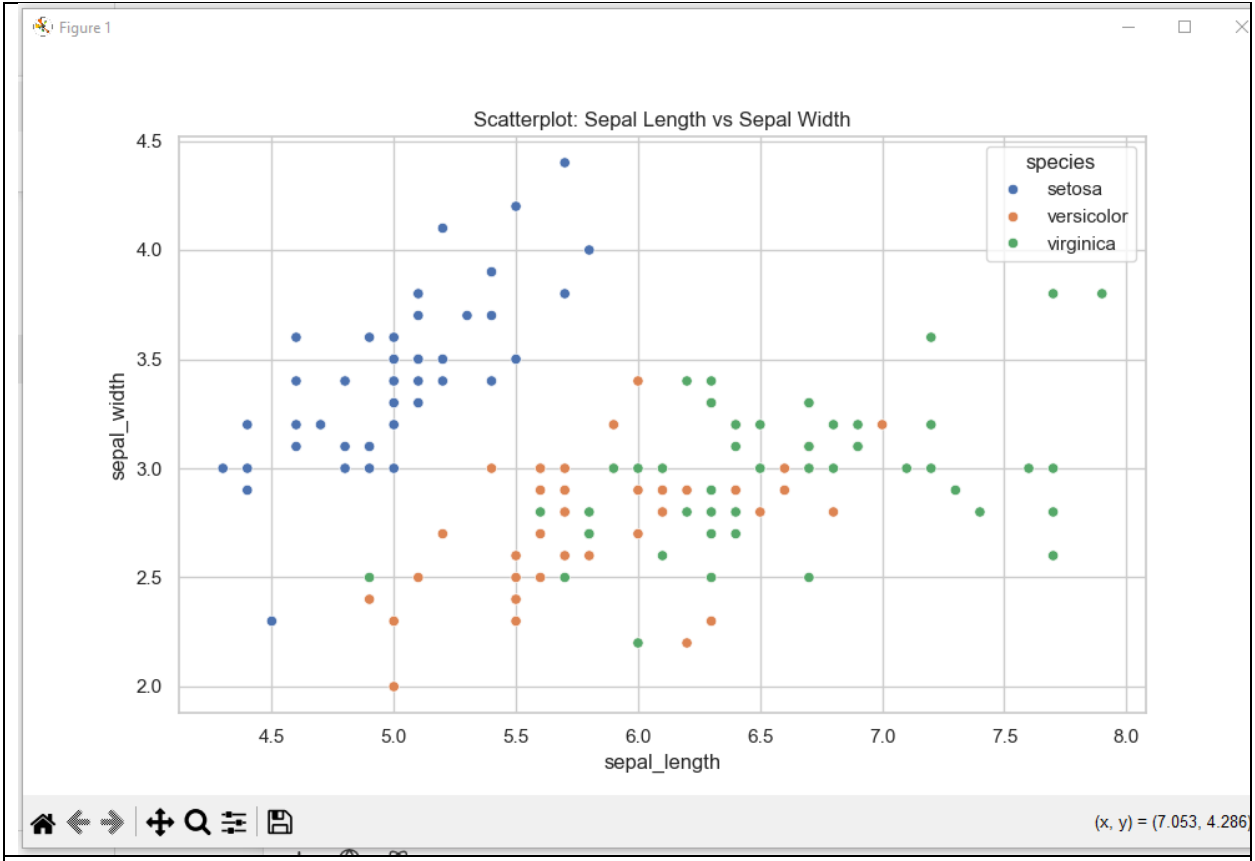
Task 1

Code

```
# Task 1: Exploring and Visualizing a Simple Dataset
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

def main():
    # 1. Load the dataset using seaborn
    # Seaborn has built-in datasets, so we don't need an external CSV file for
this task.
    iris = sns.load_dataset('iris')

    print("--- Task 1: Iris Dataset Analysis ---")

    # 2. Display dataset structure
    print("\nDataset Shape:")
    print(iris.shape)

    print("\nDataset Columns:")
    print(iris.columns)

    print("\nFirst 5 Rows (Head):")
    print(iris.head())

    # 3. Create Basic Visualizations
    # Set a generic style for the plots
    sns.set_theme(style="whitegrid")

    # A. Scatterplot to analyze relationships between variables
    # Analyzing relationship between Sepal Length and Sepal Width, colored by
species
    plt.figure(figsize=(10, 6))
    sns.scatterplot(x='sepal_length', y='sepal_width', hue='species',
data=iris, palette='deep')
    plt.title('Scatterplot: Sepal Length vs Sepal Width')
    plt.savefig('task1_scatterplot.png') # Saving the figure
```

```
    plt.show()

    # B. Histogram to examine data distribution
    # Analyzing the distribution of Petal Length
    plt.figure(figsize=(10, 6))
    sns.histplot(iris['petal_length'], kde=True, bins=20, color='purple')
    plt.title('Histogram: Distribution of Petal Length')
    plt.savefig('task1_histogram.png')
    plt.show()

    # C. Box plot to detect outliers and spread of values
    # Analyzing spread of all numerical features
    plt.figure(figsize=(10, 6))
    sns.boxplot(data=iris)
    plt.title('Boxplot: Spread of Values (Detection of Outliers)')
    plt.savefig('task1_boxplot.png')
    plt.show()

    print("Visualizations saved successfully.")

if __name__ == "__main__":
    main()
```

Result



Description

Scatterplot: Sepal Length vs Sepal Width

Description

This scatterplot illustrates the relationship between **Sepal Length** and **Sepal Width** in the Iris dataset.

- The **X-axis** represents **Sepal Length (cm)**.
- The **Y-axis** represents **Sepal Width (cm)**.

The data points are grouped and colored according to flower species:

Setosa (Blue)

- Generally has a shorter sepal length.
- Has a relatively larger sepal width.
- Data points are closely clustered together.

Versicolor (Orange)

- Has a moderate sepal length.
- Has a moderate sepal width.
- Lies between Setosa and Virginica in terms of measurements.

Virginica (Green)

- Generally has the longest sepal length.
- Shows a wider range of sepal widths.
- Data points are more spread out compared to Setosa.

Conclusion

The scatterplot shows that the **Setosa** species is clearly separated from the other two species based on sepal measurements. In contrast, **Versicolor** and **Virginica** exhibit some overlap, indicating that sepal length and sepal width alone may not be sufficient to completely distinguish between these two species.

Task 2

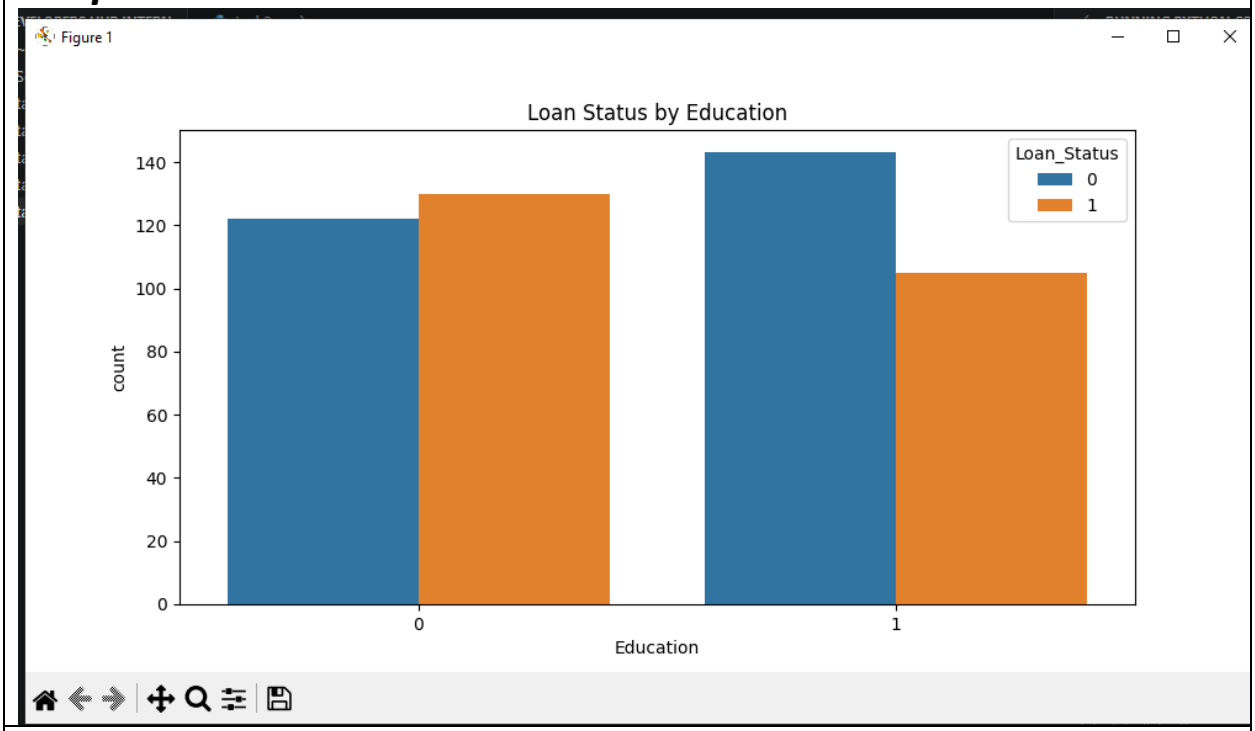
Code

```
# Task 2: Credit Risk Prediction
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
import seaborn as sns
import matplotlib.pyplot as plt

def main():
```

```
    print("--- Task 2: Credit Risk Prediction ---")

    # 1. Create Synthetic Data (Mimicking a Loan Dataset)
    # In a real scenario, you would load: df = pd.read_csv('loan_data.csv')
    data = {
        'ApplicantIncome': np.random.randint(2000, 10000, 500),
        'LoanAmount': (np.random.rand(500) * 200 + 50).round(2),
        'Education': np.random.choice(['Graduate', 'Not Graduate'], 500),
        'Credit_History': np.random.choice([0.0, 1.0], 500, p=[0.15, 0.85]),
        # Target variable (1 = Default/No, 0 = Approved/Yes - Logic varies,
here 0=Good, 1=Bad)
        'Loan_Status': np.random.choice([0, 1], 500)
    }
    df = pd.DataFrame(data)

    # Introduce some missing values to demonstrate handling skills
    df.loc[10:15, 'LoanAmount'] = np.nan
    df.loc[20:22, 'Credit_History'] = np.nan

    print("\nOriginal Data Head:")
    print(df.head())

    # 2. Handle Missing Data Appropriately
    # Fill numerical missing values with the mean
    df['LoanAmount'].fillna(df['LoanAmount'].mean(), inplace=True)
    # Fill categorical missing values with the mode
    df['Credit_History'].fillna(df['Credit_History'].mode()[0], inplace=True)
    print("\nMissing Values after handling:")
    print(df.isnull().sum())

    # 3. Encode Categorical Features
    # Convert 'Education' to binary (0 and 1)
    df['Education'] = df['Education'].map({'Graduate': 1, 'Not Graduate': 0})

    # 4. Visualize Key Features
    plt.figure(figsize=(10, 5))
    sns.countplot(x='Education', hue='Loan_Status', data=df)
    plt.title('Loan Status by Education')
    plt.savefig('task2_education_viz.png')
    plt.show()

    # 5. Train Classification Model
    # Define features (X) and target (y)
    X = df.drop('Loan_Status', axis=1)
    y = df['Loan_Status']
```

```
    # Split into train and test sets (80% train, 20% test)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    # Initialize and train Logistic Regression
    model = LogisticRegression(max_iter=1000)
    model.fit(X_train, y_train)

    # 6. Evaluate Model
    predictions = model.predict(X_test)

    print("\n--- Model Evaluation ---")
    print(f"Accuracy: {accuracy_score(y_test, predictions):.2f}")
    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, predictions))
    print("\nClassification Report:")
    print(classification_report(y_test, predictions))

if __name__ == "__main__":
    main()
```

Output



Description

Loan Status by Education

Description

This bar chart shows the relationship between **Education** and **Loan Status**.

- The **X-axis** represents the education level:
  - **0 = Not Graduate**
  - **1 = Graduate**
- The **Y-axis** represents the number of applicants.
- The bars are grouped according to **Loan Status**:
  - **0 = Loan Not Approved**
  - **1 = Loan Approved**

Observations

- Among **Non-Graduates (Education = 0)**, the number of approved loans is slightly higher than the number of rejected loans.
- Among **Graduates (Education = 1)**, the number of rejected loans is higher than the number of approved loans.
- Graduates have a larger total number of loan applications compared to non-graduates.

Conclusion

The chart indicates that loan approval varies with education level. Non-graduates show a slightly higher number of approved loans, while graduates have a higher number of rejected loans. However, education alone may not be sufficient to determine loan approval, as other factors such as income, credit history, and loan amount may also influence the decision.

Task 4

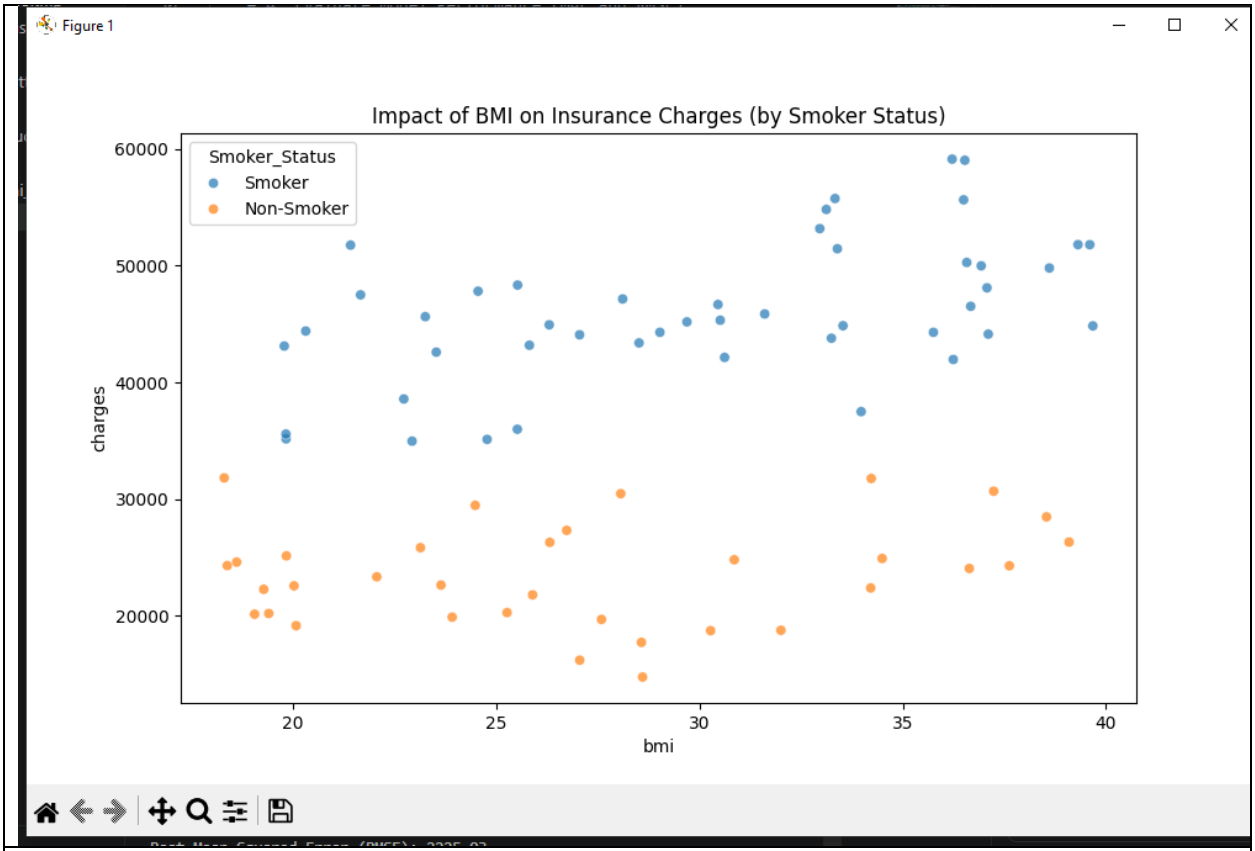
Code

```
# Task 4: Predicting Insurance Claim Amounts
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error

def main():
    print("--- Task 4: Predicting Insurance Claim Amounts ---")

    # 1. Create Synthetic Data (Mimicking Medical Cost Personal Dataset)
    np.random.seed(42)
    n_samples = 400

    data = {
```

```
        'age': np.random.randint(18, 70, n_samples),
        'bmi': np.random.uniform(18, 40, n_samples),
        'smoker': np.random.choice(['yes', 'no'], n_samples),
        # Charges are heavily influenced by Age, BMI, and Smoking status
        'charges': np.random.uniform(2000, 50000, n_samples)
    }
    df = pd.DataFrame(data)

    # Add logic to make 'charges' dependent on features so the model actually
learns something
    # If smoker, charges are higher
    smoker_penalty = df['smoker'].apply(lambda x: 20000 if x == 'yes' else 0)
    df['charges'] = (df['age'] * 300) + (df['bmi'] * 400) + smoker_penalty +
np.random.normal(0, 2000, n_samples)

    print("\nData Sample:")
    print(df.head())

    # 2. Encode Categorical Features (Smoker status)
    # Convert 'smoker' yes/no to 1/0
    df['smoker'] = df['smoker'].map({'yes': 1, 'no': 0})

    # 3. Train Linear Regression Model
    X = df[['age', 'bmi', 'smoker']]
    y = df['charges']

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

    model = LinearRegression()
    model.fit(X_train, y_train)

    # Predict
    y_pred = model.predict(X_test)

    # 4. Evaluate Model Performance (MAE and RMSE)
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

    print("\n--- Model Evaluation ---")
    print(f"Mean Absolute Error (MAE): {mae:.2f}")
    print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")

    # 5. Visualize Impact of Features on Charges
```

```
    # Visualization 1: Impact of BMI and Charges colored by Smoker
    plt.figure(figsize=(10, 6))
    # We reconstruct a temporary dataframe for plotting purposes using original
boolean mapping
    plot_df = X_test.copy()
    plot_df['charges'] = y_test
    plot_df['Smoker_Status'] = plot_df['smoker'].map({1: 'Smoker', 0: 'Non-
Smoker'})

    sns.scatterplot(x='bmi', y='charges', hue='Smoker_Status', data=plot_df,
alpha=0.7)
    plt.title('Impact of BMI on Insurance Charges (by Smoker Status)')
    plt.savefig('task4_bmi_impact.png')
    plt.show()

    # Visualization 2: Actual vs Predicted Charges
    plt.figure(figsize=(10, 6))
    plt.scatter(y_test, y_pred, color='blue')
    plt.plot([y.min(), y.max()], [y.min(), y.max()], 'k--', lw=2) # Diagonal
line
    plt.xlabel('Actual Charges')
    plt.ylabel('Predicted Charges')
    plt.title('Actual vs Predicted Insurance Charges')
    plt.savefig('task4_prediction_accuracy.png')
    plt.show()

if __name__ == "__main__":
    main()
```

Output



Description

Impact of BMI on Insurance Charges (by Smoker Status)

Description

This scatter plot illustrates the relationship between **Body Mass Index (BMI)** and **Insurance Charges**, categorized by **Smoker Status**.

- The **X-axis** represents **BMI (Body Mass Index)**.
- The **Y-axis** represents **Insurance Charges**.
- Each point represents an individual.
- **Blue points** indicate **Smokers**.
- **Orange points** indicate **Non-Smokers**.

Observations

- Smokers generally have much higher insurance charges than non-smokers.

- As BMI increases, insurance charges tend to increase, especially for smokers.
- Non-smokers have comparatively lower and more consistent insurance charges across different BMI values.
- The highest insurance charges are mostly associated with smokers having higher BMI values.

Conclusion

The graph suggests that both **BMI** and **smoking status** influence insurance charges. Smokers incur significantly higher insurance costs than non-smokers, and higher BMI is associated with increased charges. Smoking appears to have the strongest impact on insurance expenses.